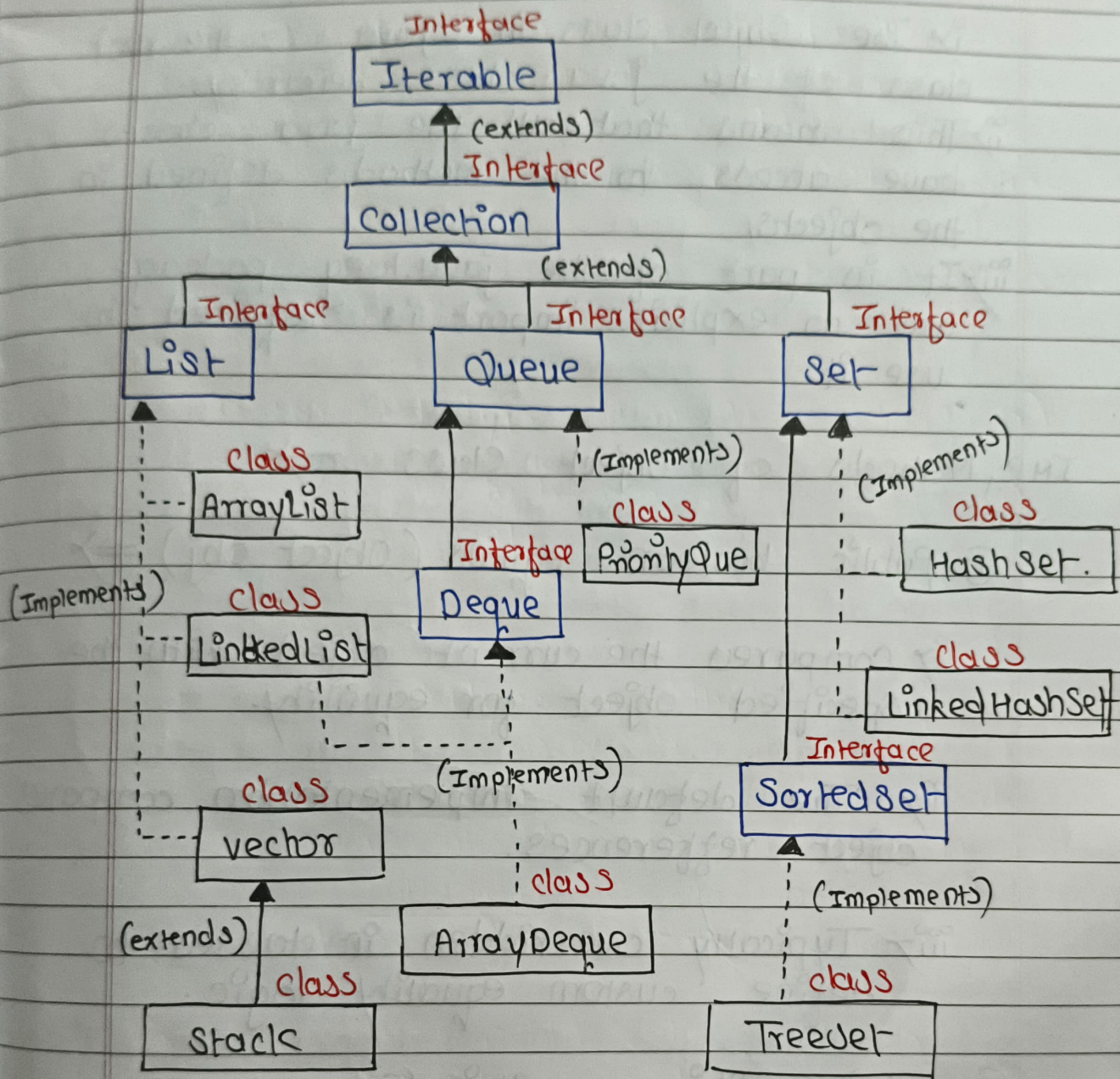
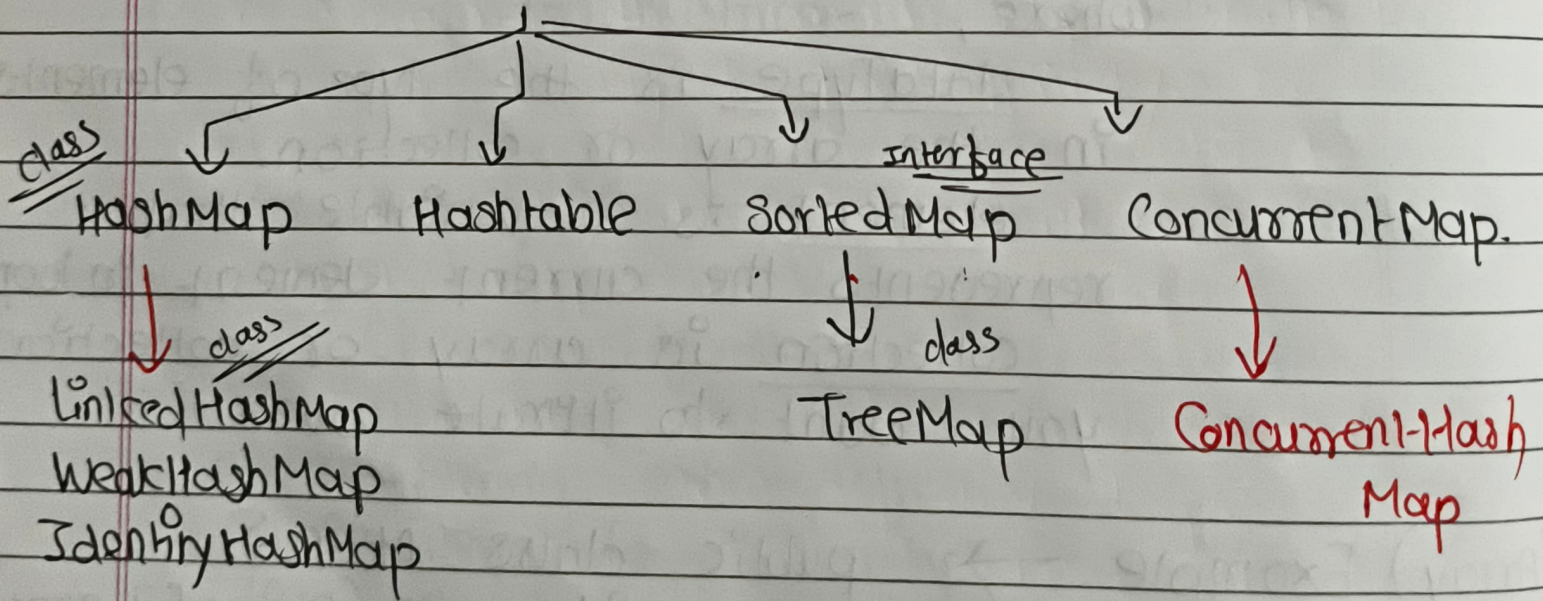


* Collection Hierarchy in Java ⇒



The Collection Framework provides a structured hierarchy to manage and manipulate groups of objects. It is part of java.util package. It includes interfaces, classes, and methods to handle collection of data.

Map (Separate Interface)



for Revision watch Engineering Digest video }



* Collections :->

The collection in Java is a framework that provides an architecture to store and manipulate the group of objects.

Java Collections can achieve all the operations that you ~~per~~ perform on a data such as searching, sorting, insertion, manipulation and deletion.

Java collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

* What is collection ?

A collection represents a single unit of objects, i.e. a group.

* What is framework ?

A framework provides a ready-made structure of classes and interfaces for building software applications.

Key points :->

i) It provide readymade Architecture.

ii) It represents a set of classes & interfaces.

iii) It is Optional.

* What is Collection frameworks ?

The collection frameworks represents a unified architecture for storing and manipulating a group of objects. It enhances code efficiency and readability by offering various data structures, including arrays, linked lists, trees, and hash tables, etc.

* Advantages of the Java collection framework

The java collections framework offers significant advantages that enhanced development practices, code quality and application performance.

- ① Reusability
- ② Speed
- ③ Reduces Efforts to Design New APIs.

* Hierarchy of Collection frameworks

The java.util package contains all the classes & interface for the Collection frameworks

{ take a look at Hierarchy }
{ drawn before }

* Collection vs Collections in Java :->

In java, Collection and Collections are two different things, but they are often confused due to their similar names.

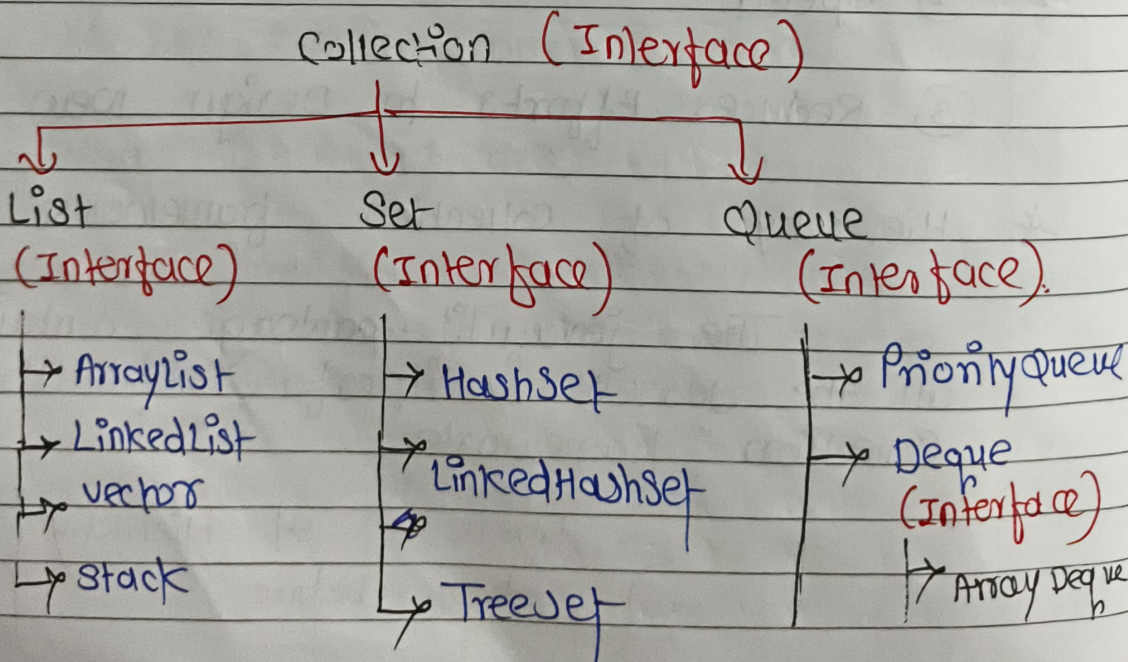
① Collection (Interface) :->

i) Collection is not interface in the Java Collection framework.

ii) It is a part of java.util package.

iii) It represents a group of objects known as elements.

iv) It extends the Iterable interface.



* Methods in Collection Interface :->

① boolean add (E e);

// Adds an element to the collection.

② boolean remove (Object o);

// Removes a specified element.

③ boolean contains (Object o);

// checks if an element exists.

④ int size();

// Return the no. of elements

⑤ boolean isEmpty();

// checks if collection is empty

⑥ void clear();

// Remove all elements.

⑦ Iterator <E> iterator();

// Returns an iterator for the collection

Now let's see code for this methods

DATE / / PAGE

* * * Java Program :-

```
import java.util.*;
```

```
public class CollectionMethodDemo {
```

```
    public static void main(String[] args) {  
        // creating a collection (ArrayList)  
        Collection<String> collection = new ArrayList  
        <>();
```

```
// ① add (E e) → Adds an element - to the  
collection.
```

```
collection.add("Java");  
collection.add("Python");  
collection.add("C++");
```

```
System.out.println("Collection: " + collection);
```

```
// Output : [Java, Python, C++]
```

```
// ② remove (Object o) → Removes a specified  
element
```

```
collection.remove("Python");  
System.out.println("After removing Python:"  
+ collection);
```

```
// Output : [Java, C++]
```

```
// ③ contains (Object o) → checks if an  
element exists
```

```
System.out.println("contains Java ?:"  
+ collection.contains("Java"));
```

```
Output : true.
```

// ④ size() → Returns the number of elements.

System.out.println("Size of collection: " + collection.size());

// output : 2

// ⑤ isEmpty() → Checks if collection is empty

System.out.println("Is collection empty? " + collection.isEmpty());

// output : false.

// ⑥ Iterator() → Iterating over the collection.

System.out.println("Iterating over elements");

Iterator<String> itr = collection.iterator();
while (itr.hasNext()) {

System.out.print(itr.next() + " ");
}

// output : Java C++

// ⑦ Clear() → Removes all elements

collection.clear();

System.out.println("In collection after clear() : " + collection);

Output : []

}

}

② Collections (Class) :->

- i) Collections is a utility class in Java.util package.
- ii) It provides static methods for sorting, searching, reversing, and synchronizing collections.
- iii) It cannot be instantiated since it is a final class.

Methods in Collections Class :-> (with program)

```
import java.util.*;
```

```
public class CollectionsMethodsDemo {
```

```
    public static void main (String[] args) {
```

```
        // Creating a List.
```

```
        List<Integer> numbers = new ArrayList<>(  
            Arrays.asList(5, 2, 8, 1, 9, 5));
```

OR

```
        // Creating a List.
```

```
        List<Integer> numbers = new ArrayList<>();
```

```
        number.add(5);
```

```
        number.add(2);
```

```
        number.add(8);
```

```
        number.add(1);
```

```
        number.add(9);
```

// ① Collections.sort(List<T> list) → Sorts the list.

```
Collections.sort(numbers);
System.out.println("Sorted List: " + numbers);
Output : [1, 2, 5, 5, 8, 9]
```

// ② Collections.reverse(List<T> list) → Reverse the order.

```
Collections.reverse(numbers);
System.out.println("Reversed List: " + numbers);
Output : [9, 8, 5, 5, 2, 1]
```

// ③ Collections.shuffle(List<T> list) → Random Shuffle

```
Collections.shuffle(numbers);
System.out.println("Shuffled List: " + numbers);
Output : (Random) [5, 9, 2, 1, 8, 5]
```

// ④ Collections.max(Collection<T> c) → find max element.

```
int maxElement;
maxElement = Collections.max(numbers);
System.out.println("Max Element: " + maxElement);
Output : 9.
```

// ⑤ Collections.min(Collection<T> c) → find min element

```
int minElement;
minElement = Collections.min(numbers);
System.out.println("Min Element: " + minElement);
Output : 1
```

Collection

Collections

i) It is interface

i) It is class

ii) Defined a group of elements

ii) Provides utility functions.

iii) Java Collection Framework

iii) java.util package

iv) Contains Methods are add(), remove(), size().

iv) Contains Methods are sort(), reverse(), shuffle().

v) Implemented by list, set, Queue.

v) Not implemented, only used.

* Key Interfaces in Java Collection

① Collection (Interface) →

Description →

The root interface of the java collection framework.

i) List (Interface) →

Ordered collection (ArrayList, LinkedList, Vector).

ii) Set (Interface) →

Unordered collection that doesn't allow duplicates (HashSet, LinkedHashSet, TreeSet).

iii) Queue (Interface) →

FIFO (first in, first out) collection (PriorityQueue, LinkedList).

iv) Deque (Interface) →

Double-ended queue (ArrayDeque, LinkedList).

v) Map (Interface) →

key value pair collection (HashMap, TreeMap, LinkedHashMap).

To instantiate the list interface, we must use

List <data-type> list1 = new ArrayList();

List <data-type> list2 = new LinkedList();

List <data-type> list3 = new Vector();

List <data-type> list4 = new Stack();

① ① List (Interface) → (Extended collection Interface)

A list is an ordered collection that allows duplicate elements.

Implementations of list →

ix ArrayList ⇒ (class) (Implements list)

The ArrayList class implements the list interface. It uses a dynamic array to store the duplicate element of ~~different~~ same data types.

The ArrayList class maintains the insertion order and is non-synchronized. The elements stored in the ArrayList class can be randomly accessed.

Example → import java.util.*;
class TestCollection {
public static void main(String args[]) {

// Creating
arraylist.

ArrayList<String> list = new ArrayList<String>();

list.add("Ravi");

list.add("Vijay");

list.add("Ankit");

list.add("Ravi");

// Adding object in
arraylist

output →

Ravi

Vijay

Ankit

Ravi.

// Traversing list through iterator.

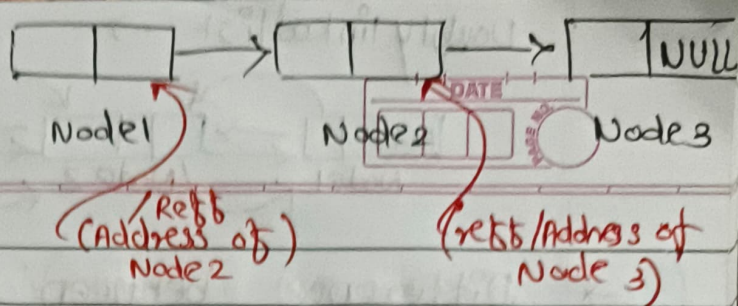
Iterator itr = list.iterator();

while (itr.hasNext()) {

System.out.println(itr.next());

}

singly linked list →



OR

```
System.out.println(list);  
// Output : [Ravi, vijay, Ankit, Ravi]
```

ii) LinkedList ⇒ (class) (Implement- list)

LinkedList class implements the collection interface. It uses a doubly linked list internally to store the elements. It uses a doubly linked list internally to store the elements.

It can store the duplicate elements. It maintains the insertion order and it not synchronized. In LinkedList, the manipulation is fast because no shifting is required.

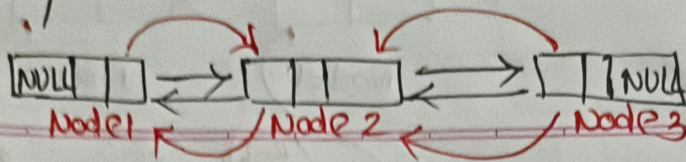
```
Example → import java.util.*;  
public class TestJavaCollection2 {  
    public static void main(String args[]) {  
        LinkedList<String> list = new LinkedList<String>();  
        list.add("Ravi");  
        list.add("Ankit");  
        list.add("Ravi");  
        list.add("Ajay");
```

```
System.out.println(list);
```

```
// Output : [A, B, C]
```

```
[Ravi, Ankit, Ravi,  
Ajay]
```

Doubly linked list →



DATE

PAGE NO.

* Difference between ArrayList and LinkedList.

ArrayList

LinkedList.

i) Uses a dynamic array internally.

i) Uses a doubly linked list internally.

ii) Less memory usage (store only data)

ii) More memory usage (stores data + pointer to next/previous nodes)

iii) Fast Random access using index

iii) Needs to traverse from the head.

iv) Faster due to contiguous memory storage

iv) Slower due to pointer based traversal.

v) When frequent read operations are needed.

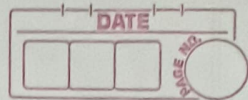
v) When frequent insertion / deletions are needed.

* When to use what?

i) Use ArrayList when you need fast random access and frequent read operations.

ii) Use LinkedList when you need frequent insertions / deletions (especially at the beginning or middle).

(legacy class)



iii) Vector $\Rightarrow$ (class) (implements list)

Vector class uses a dynamic array to store the data elements.

It is similar to ArrayList. However, it is synchronized and contains many methods that are not the part of collection framework.

Example $\Rightarrow$

```
import java.util.*;  
public class TestJavaCollection3 {  
    public static void main(String args[]) {  
        Vector<String> v = new Vector<String>();
```

```
        v.add("Ayush");  
        v.add("Ankit");  
        v.add("Aryan");  
        v.add("Prathmesh");
```

```
        Iterator<String> itr = v.iterator();
```

```
        while (itr.hasNext()) {
```

```
            System.out.println(itr.next());
```

```
        }  
    }  
}
```

iv) Stack $\Rightarrow$ (class) (extends Vector)

The stack is the subclass of Vector. It implements the last-in-first-out data structure, i.e. stack.

The stack contains all of the methods of Vector class and also provides its methods like boolean push(), boolean peek(), boolean push(Object o), which defines its properties.

Example $\Rightarrow$

```
import java.util.*;
public class TestJavaCollection4 {
    public static void main(String args[]) {
```

```
        Stack<String> stack = new Stack<String>();
```

```
        stack.push("Ayush");
```

```
        stack.push("Ankit");
```

```
        stack.push("Ashish");
```

```
        stack.push("Amit");
```

```
        stack.pop();
```

```
        Iterator<String> itr = stack.iterator();
```

```
        while (itr.hasNext()) {
```

```
            System.out.println(itr.next());
```

```
        }
    }
```


DATE PAGE NO.

Example :-> (PriorityQueue)

```
import java.util.*;
public class TestJavaCollections {
    public static void main (String args[]) {
        PriorityQueue<String> queue = new PriorityQueue<String> ();

        queue.add ("Amit Sharma");
        queue.add ("Ankit Thora");
        queue.add ("Jai Shankar");
        queue.add ("Raju");

        System.out.println ("head " + queue.element());
        System.out.println ("head " + queue.peek());
        System.out.println ("Iterating the queue element");

        Iterator itr = queue.iterator();

        while (itr.hasNext()) {
            System.out.println(itr.next());
            queue.remove();
            queue.poll();
        }

        System.out.println ("After removing two elements");
    }
}
```

```

Iterator<String> itr2 = queue.iterator();
while (itr2.hasNext()) {
    System.out.println(itr2.next());
}
}
}

```

Output :->

head : Amit Sharma
 head : Amit Sharma
 iterating the queue elements:
 Amit Sharma
 Raju Ankit Thorat
 Jaishankar
~~Amit Thorat~~ Raju
 After removing two elements:
~~Raju~~ Jaishankar
~~Ankit Thorat~~ Raju

i) element() and peek() are used to retrieve the head (smallest element) of the queue.

ii) Both methods return the same element, but element() throws an exception if the queue is empty, while

}

DATE PAGE NO.

③ ~~iii~~ Deque $\Rightarrow$ (Interface) (extends Queue Interface)

Deque interface extends the Queue interface. In Deque, we can remove and add the elements from both the side.

Deque stands for a double-ended queue which enables us to perform the operations at both the ends.

Deque can be instantiated as.

```
Deque d = new ArrayDeque();
```

ix ArrayDeque (class) $\Rightarrow$ (Implements Deque)

ArrayDeque class implements the Deque interface. It facilitates us to use the Deque. Unlike queue, we can add or delete the elements from both the ends.

ArrayDeque is faster than ArrayList and Stack and has no capacity restrictions.

ArrayDeque is faster than stack for LIFO operations.

It is Resizable, No capacity Restrictions

Supports FIFO (queue) & LIFO (stack) operations.

Example :->

```
import java.util.*;

public class TestJavaCollection6 {
    public static void main(String[] args) {
        // creating Deque and adding elements.
        Deque<String> deque = new ArrayDeque<String>();
        deque.add("Gautam");
        deque.add("Karan");
        deque.add("Ajay");

        // Traversing elements
        for (String str : deque) {
            System.out.println(str);
        }
    }
}
```

Output :->

Gautam
Karan
Ajay.

③ ~~3~~ Set (Interface) $\Rightarrow$ (Extends collection Interface)

Set interface in Java is present in java.util package. It extends the collection interface.

It represents the unordered set of elements which doesn't allow us to store the duplicate items.

We can store at most one null value in set. Set is implemented by HashSet, LinkedHashSet, and TreeSet.

Set can be instantiated as :

```
Set<data-type> s1 = new HashSet<data-type>();
Set<data-type> s2 = new LinkedHashSet<data-type>();
Set<data-type> s3 = new TreeSet<data-type>();
```

is HashSet (class) $\Rightarrow$ (Implements Set)

HashSet class implements Set interface. It represents the collection that uses a hash table for storage.

Hashing is used to store the elements in the HashSet. It contains unique items.

Example :->

```
import java.util.* ;

public class TestJavaCollection {

    public static void main(String args[]) {

        // Creating HashSet and adding elements.
        HashSet<String> set = new HashSet<String>();

        set.add("Ravi");
        set.add("Vijay");
        set.add("Ravi");
        set.add("Ajay");

        // Traversing elements.
        Iterator<String> itr = set.iterator();
        while (itr.hasNext()) {

            System.out.println(itr.next());
        }
    }
}
```

Output ->

Vijay
Ravi
Ajay

DATE / / PAGE No.

ii) LinkedHashSet (class) :->

(it extends HashSet & implements Set)

LinkedHashSet class represents the Linkedlist implementation of Set interface.

It extends the HashSet class and implements Set interface. Like HashSet, it also contains unique elements.

It maintains the insertion order and permits null elements.

Example :->

```
import java.util.*;

public class TestJavaCollections {

    public static void main(String args[]) {

        LinkedHashSet<String> set = new
            LinkedHashSet<String>();

        set.add("Ravi");
        set.add("Vijay");
        set.add("Ravi");
        set.add("Ajay");

        Iterator<String> itr = set.iterator();

        while (itr.hasNext()) {
            Syso(itr.next());
        }
    }
}
```

Output :->

Ravi
Vijay
Ajay.